

9.1 Preference Data and Reward Models

Bradley–Terry on a pair. Chance is $\log 2$. Best-of- n uses the RM without PPO.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

SFT copies demonstrations. **Preferences** say which of two replies is better when both are fluent. A **reward model (RM)** is a classifier of that comparison. Week 9.2 will use the RM inside RL; Week 9.3 will skip it.

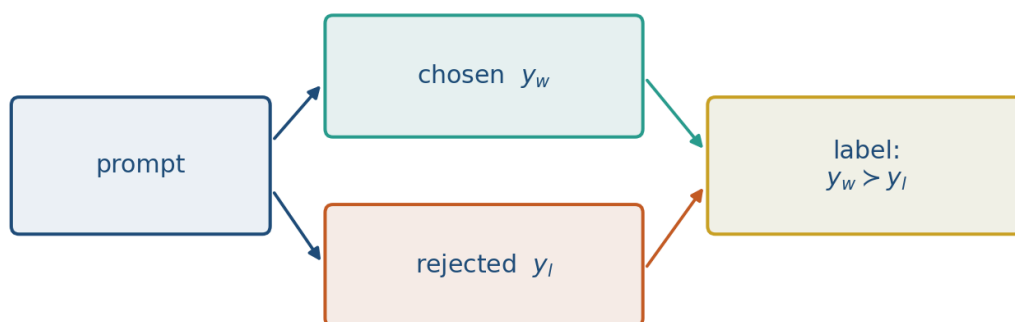
First time this method appears. A **reward model (RM)** scores completions from **preference pairs** (chosen vs rejected).

What. Humans (or a teacher) pick y_w over y_l for prompt x . Bradley–Terry turns that into a logistic loss on $r(x, y)$. **Why.** SFT copies a single answer. Preferences say “this is better than that” without writing a scalar label by hand. **Architecture.** Usually the base LM plus a scalar head, or a small classifier on the last token. Best-of- n : sample n , pick $\arg \max r$, no PPO yet. **How.** Collect pairs. Train r . Chance loss is $\log 2$. Lab 9 is a tiny head. **Formula.** $p(y_w > y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$. Loss $-\log \sigma(\Delta r)$. **Tradeoffs.** + Pairwise data is easier than absolute scores. – Reward hacking (next notes); RM error becomes policy error; best-of- n is compute at decode.

1. Chosen versus rejected

A human (or a strong teacher model) sees a prompt x and two completions y_w (**chosen**, winner) and y_l (**rejected**, loser). The label is a pairwise order, not a numeric grade. Ties and multi-way ranks exist; this course uses pairs.

One preference row is a comparison, not a score out of 100



Collection is the bottleneck: instructions must be clear, raters disagree, and the pair set can encode rater bias. If both y_w and y_l are wrong, the RM still learns a relative order among mistakes.

2. The reward model

Initialize from the SFT policy (or its encoder). Score a pair with a scalar head $r_\phi(x, y)$. Bradley–Terry / logistic loss:

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

The RM wants a higher number on the chosen reply. Lab 9 will fit this on toy scores. At test time you can rank n samples from the policy by r_ϕ (**best-of- n**) without any RL.

A reward model is a classifier of “better completion,” trained on pairs



If the two rewards are equal, the argument of σ is 0 and $\sigma(0) = 1 / 2$, so $\mathcal{L}_{\text{RM}} = \log 2 \approx 0.693$. That is chance: the RM has not ordered the pair.

Three Δ values and the logistic loss

Δ values and the logistic loss" src="graphics/9.1-preference-rewards/rm-numeric.png" />

$\sigma(\Delta) = (1 + e^{-\Delta})^{-1}$. Large positive $\Delta \rightarrow$ loss near 0; large negative Δ (chosen scored **below** rejected) $\rightarrow$ loss near ∞ . The RM is a pairwise logistic classifier, not a calibrated “helpfulness out of 10.”

Best-of- n already uses this RM and is a competitive baseline (AlpacaFarm). It is not PPO. You sample n from SFT and return the RM’s favorite. PPO (next note) trains a new π_θ .

3. What the number is not

r_ϕ is not truth, safety, or “helpfulness” in the abstract. It is a fit to the raters you hired, on the prompts you wrote. Reward hacking: the policy learns to please r_ϕ with verbose, sycophantic, or empty-safe text. That is why RLHF adds a KL penalty to the SFT model (next note), and why you keep a held-out preference set.

Held-out preference **accuracy** is how often $r_\phi(x, y_w) > r_\phi(x, y_l)$ on pairs the RM did not train on. Report that. Do not report only training loss.

4. Teaching this note

About **30–35 minutes** at the board: one pair, write $\sigma(\Delta r)$, compute $\sigma(0)$, then best-of- n cost. Play Karpathy's **RLHF / comparison** appendix **21:05–25:43**. Lab 9 section 1 is the toy logistic RM.

Compute $\log 2$ once as a class so “0.693 at init” is a number people can check in Lab 9, not a magic constant.

5. Worked example

Rewards $r_w = 2.0$, $r_l = 0.0$. Difference 2. $\sigma(2) = 1 / (1 + e^{-2}) \approx 0.881$. Loss $-\log 0.881 \approx 0.127$.

If $r_w = r_l = 1.7$ always, $\Delta = 0$, $\sigma(0) = 1 / 2$, $\mathcal{L} = -\log(1 / 2) = \log 2 \approx 0.693$ nats per pair. A random RM sits here.

Best-of- n : sample $n = 8$ completions from π_{SFT} , score each with r_ϕ , return $\arg \max_i r_\phi(x, y_i)$. Extra cost: 8 policy forwards plus 8 RM forwards (the RM is usually cheaper). No PPO. The distribution is “SFT, then pick the RM’s favorite,” not a new π_θ .

Init from SFT: the RM already speaks the same dialect as the completions it must score. A raw pretrained LM does not format like an assistant, so its scalar head starts on the wrong geometry.

If $r_w = 0$, $r_l = 2$, $\Delta = -2$, $\sigma(-2) \approx 0.119$, loss $-\log 0.119 \approx 2.13$. The RM is **wrong** on that pair; that is a large gradient. Lab 9 should print both the easy pair and a flipped pair.

6. Where students get stuck

- Treating r_ϕ as a grade out of 10. Only **differences** enter the loss.
 - Forgetting $\sigma(0) = 1 / 2$ when the two rewards match.
 - Calling best-of- n “RLHF.” It uses the RM; it does not train a policy with PPO.
-

7. Video

[Karpathy — Intro to Large Language Models](#). Play the **RLHF / preference** section **21:05–25:43** (comparisons, labeling, RLHF). Pause on “it is easier to compare than to write”: that is why we collect pairs, not essays. PPO details wait for note **9.2**.

8. Practice

1. If $r(x, y_w) = r(x, y_l)$ always, what is $\mathcal{L}_{\text{RM}}$?
2. Why initialize the RM from SFT instead of from a raw pretrained LM?

3. Best-of- n uses the RM without PPO. What extra cost did you pay at inference?
4. Compute $\sigma(r_w - r_l)$ and the RM loss for $r_w = 1, r_l = -1$. ($\sigma(2) \approx 0.881$.)
5. You score $n = 4$ samples. Two share the top RM score. How do you break the tie in a reproducible demo, and what did the RM fail to do?